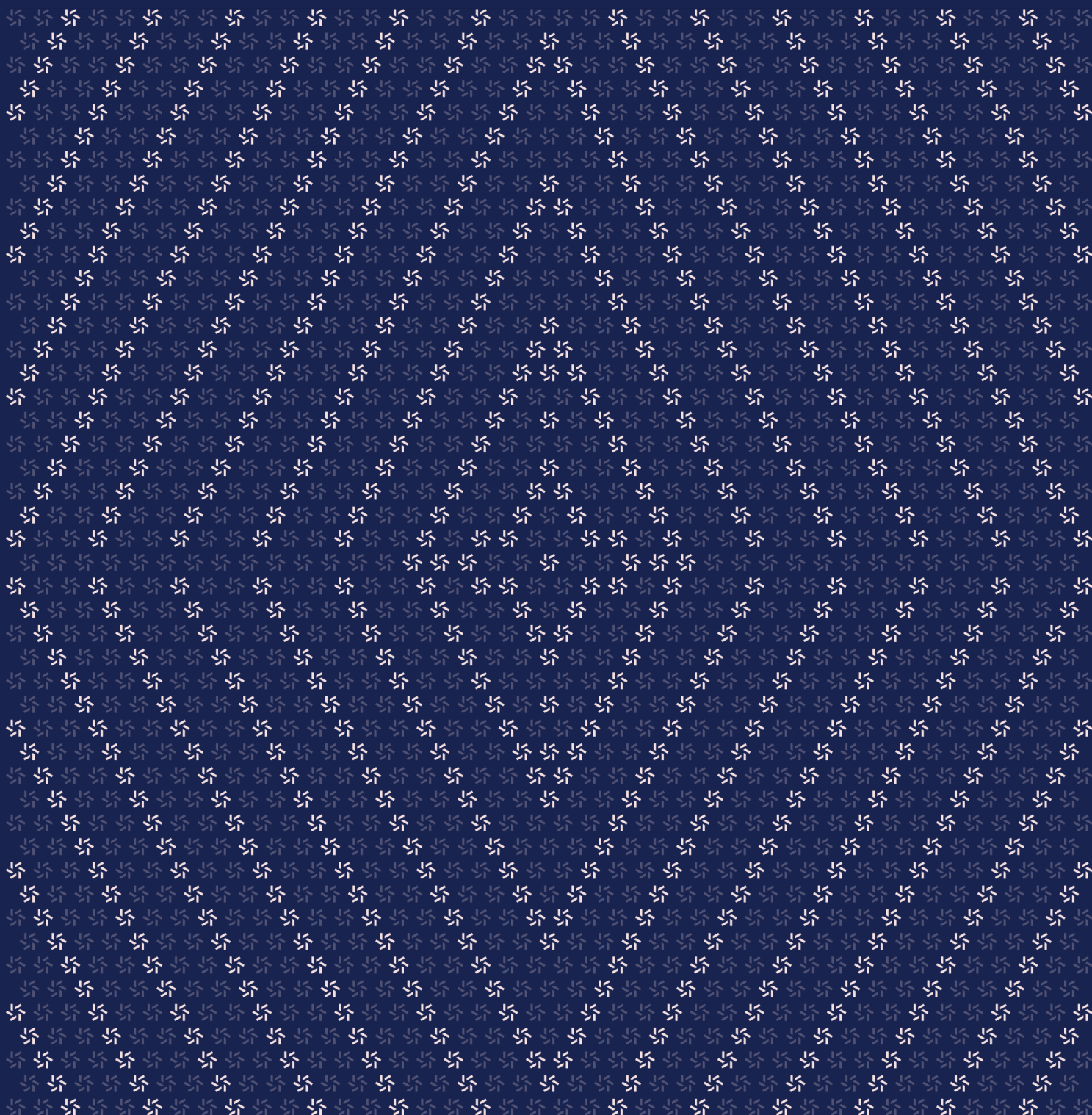


March 13, 2026

Deposit Wallet

Smart Contract Security Assessment



Contents

About Zellic	4
---------------------	----------

1. Overview	4
--------------------	----------

1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6

2. Introduction	6
------------------------	----------

2.1. About Deposit Wallet	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10

3. Detailed Findings	10
-----------------------------	-----------

3.1. Session-signer authorizations are not cleared on ownership change	11
3.2. Assembly block in <code>isValidSignature</code> violates memory-safe contract	14
3.3. Unbounded timelock delay lacks on-chain safeguards	17
3.4. Operator can invoke arbitrary contracts through unvalidated proxy calls	19
3.5. Missing zero-address check in wallet initialization allows ownerless wallets	21
3.6. Test suite	23

4.	Discussion	23
4.1.	Cross-contract dependency on factory's sequential storage slots	24
4.2.	Vulnerable wallet implementations cannot be forcibly retired on chain	25
4.3.	Single batch failure in proxy reverts all batches in the transaction	26
5.	Threat Model	27
5.1.	Module: DepositWallet.sol	28
5.2.	Module: DepositWalletFactory.sol	40
6.	Assessment Results	47
6.1.	Disclaimer	48

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io and follow [@zellic_io](#) on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io.



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Polymarket from March 5th to March 9th, 2026. During this engagement, Zellic reviewed Deposit Wallet's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can a user take any on-chain action after off-chain matching that would cause on-chain settlement to fail?
 - Is the access-control model across the factory and wallet contracts correctly enforced? Specifically, can a session signer bypass the `target != address(this)` restriction or escalate privileges?
 - Are the EIP-712 signature schemes (batch execution and ownership handover) secure against replay, forgery, and cross-chain attacks? Can the session-signer signature envelope (ERC-6492 magic bytes) be spoofed or confused with a regular owner signature?
 - Could the pause and timelock mechanism be circumvented or manipulated (e.g., by resetting the timelock, retroactively changing the delay, or executing batches on a paused wallet)?
 - Is the UUPS upgrade path secure, and do the custom named storage slots remain compatible across implementations? Are there any collisions between custom slots and Solady's internal storage?
 - Could the inline assembly and compiler optimizer interactions cause memory corruption or unexpected behavior?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Off-chain infrastructure, including the operator relayer, batch simulation and validation, pause enforcement, order matching, and settlement orchestration
 - Front-end components
 - Deployment scripts, test infrastructure, mocks, and test-only libraries
 - Key custody and multi-sig configuration for privileged roles (owner, admin, operator)
 - Infrastructure relating to the project
-

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

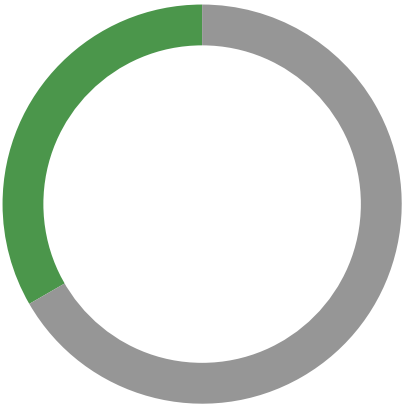
1.4. Results

During our assessment on the scoped Deposit Wallet contracts, we discovered six findings. No critical issues were found. Two findings were of low impact and the remaining findings were informational in nature.

Additionally, Zellic recorded its notes and observations from the assessment for the benefit of Polymarket in the Discussion section [\(4. 7\)](#).

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	2
Informational	4



2. Introduction

2.1. About Deposit Wallet

Polymarket contributed the following description of Deposit Wallet:

DepositWallet is a minimal, upgradeable smart-contract wallet system for managing ERC-20 and ERC-1155 token operations through signature-based batch execution.

Wallets are deployed as deterministic ERC-1967 UUPS proxies by a factory contract. Each wallet has a single owner who authorizes batches of arbitrary calls via EIP-712 signatures. Owners can delegate signing authority to time-limited session signers. An emergency pause mechanism with a configurable timelock allows the owner to recover assets and revoke approvals if a session signer is compromised.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

Finally, Zellic provides a list of miscellaneous observations that do not have security impact or are not directly related to the scoped contracts itself. These observations — found in the Discussion (4. 7) section of the document — may include suggestions for improving the codebase, or general recommendations, but do not necessarily convey that we suggest a code change.

2.3. Scope

The engagement involved a review of the following targets:

Deposit Wallet Contracts

Type	Solidity
Platform	EVM-compatible
Target	deposit-wallet
Repository	https://github.com/Polymarket/deposit-wallet ↗
Version	2d1ca79c15c9ca08ecbdeca42b78130f167b7e03
Programs	DepositWallet.sol DepositWalletFactory.sol libraries/Ownable.sol libraries/WalletLib.sol libraries/SessionSignerLib.sol interfaces/IDepositWallet.sol Errors.sol Events.sol

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of four person-days. The assessment was conducted by two consultants over the course of three calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
✈ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Filipe Alves
✈ Engineer
filipe@zellic.io ↗

Qingying Jie
✈ Engineer
qingying@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

March 5, 2026 Start of primary review period

March 5, 2026 Kick-off call

March 9, 2026 End of primary review period

3. Detailed Findings

3.1. Session-signer authorizations are not cleared on ownership change

Target	DepositWallet		
Category	Business Logic	Severity	Low
Likelihood	Low	Impact	Low

Description

Session-signer authorizations are stored in a per-signer mapping that is independent of the wallet's current owner. When ownership changes through `transferOwnership` and `completeOwnershipHandover`, the contract updates the owner but leaves all existing session signer authorizations intact.

```
function _completeOwnershipHandover(uint256 _deadline,
    bytes calldata _signature)
    internal
    virtual
{
    // [...]

    _setPendingOwner(address(0));
    _setOwner(pending);

    emit OwnershipHandoverCompleted(previousOwner, pending);
}
```

The ownership transfer only updates the owner slot — it does not invalidate or revoke any session signers that were authorized by the previous owner. Session-signer authorizations are stored in a separate mapping keyed by signer address:

```
function _setSessionSigner(address _signer, uint256 _validUntil) internal {
    bytes32 slot = _SESSION_SIGNER_SLOT;
    assembly ("memory-safe") {
        mstore(0x00, _signer)
        mstore(0x20, slot)
        sstore(keccak256(0x00, 0x40), _validUntil)
    }
}
```

During signature validation in `isValidSignature`, the contract checks whether the session signer

is authorized but does not verify any relationship between the signer and the current owner:

```
function isValidSignature(bytes32 _hash, bytes calldata _signature)
    public
    view
    override
    returns (bytes4 result)
{
    address signer = _signature.extractSessionSigner();

    if (signer == address(0)) {
        signer = owner();
    } else {
        require(
            block.timestamp < sessionSignerAuthorizedUntil(signer), SessionSignerUnauthorized()
        );
    }
    // [...]
}
```

A previous owner could authorize a long-lived session signer before initiating an ownership transfer. After the transfer completes, that session signer remains valid and can continue to sign batches that the function execute will accept. While previously authorized session signers could be discovered off chain (e.g., by indexing SessionSignerAuthorized events), there is no on-chain mechanism to enumerate them. Relying solely on off-chain tooling for this leaves a gap — an on-chain enumeration mechanism would allow the new owner to discover and revoke stale signers independently of external infrastructure.

Impact

Session signers authorized by a prior owner retain the ability to execute external calls through the wallet after ownership changes. This could allow a stale session signer to perform token transfers and approvals on behalf of the wallet without the new owner's knowledge, potentially leading to unauthorized movement of funds.

Recommendations

Consider invalidating all session-signer authorizations when ownership changes. One approach is a generation counter that increments on each ownership transfer, where session-signer validity checks include the current generation — this would automatically invalidate all authorizations from previous owners without requiring enumeration or additional storage overhead.

Remediation

This issue has been acknowledged by Polymarket.

Polymarket provided the following response to this finding:

Ownership transfers are two-step (EIP-712 signed), so they're deliberate. The new owner can revoke individual session signers themselves (since they'll know about them via off-chain event indexing). We will do this offchain for them through our relayer whenever there is an owner transfer.

3.2. Assembly block in isValidSignature violates memory-safe contract

Target	DepositWallet		
Category	Optimization	Severity	Low
Likelihood	Low	Impact	Low

Description

The isValidSignature function stores the resolved signer at a fixed memory address (0x80) and annotates the assembly block as memory - safe:

```
function isValidSignature(bytes32 _hash, bytes calldata _signature)
// [...]
{
    // [...]
    assembly ("memory-safe") {
        mstore(_ERC1271_SIGNER_MEM_SLOT, signer)
        mstore(0x40, add(mload(0x40), 0x20))
    }

    return super.isValidSignature(_hash, _signature);
}
```

However, when isValidSignature is called through the function execute, prior operations have already allocated memory past 0x80, meaning the write targets memory below the free memory pointer. This violates the memory-safe contract with the Solidity optimizer, which is allowed to assume that memory-safe blocks do not modify previously allocated memory.

The execution flow leading to this violation is as follows:

1. The function execute enters via the onlyFactory modifier, which calls the view function factory. This calls the function argsOnERC1967 of the LibClone library, which decodes immutable arguments and allocates memory at 0x80, advancing the free memory pointer.
2. Next, _batch.hash() uses abi.encode, which allocates further memory.
3. Then _hashTypedData(batchHash) uses scratch space.
4. Lastly, isValidSignature(batchDigest, _signature) writes the signer to 0x80, but the free memory pointer is well past it.

```
function execute(Batch calldata _batch, bytes calldata _signature)
    external onlyFactory {
        // ...
        // Step 1: onlyFactory → factory() → argsOnERC1967() allocates at 0x80
        // Step 2: _batch.hash() via abi.encode allocates further
        bytes32 batchHash = _batch.hash();
        bytes32 batchDigest = _hashTypedData(batchHash);

        // Step 3: isValidSignature writes signer to 0x80, below FMP
        require(
            isValidSignature(batchDigest, _signature) == ERC1271_MAGIC_VALUE,
            InvalidSignature()
        );
        // ...
    }
```

By the time `isValidSignature` executes, `0x80` sits below the free memory pointer, so the write targets previously allocated memory. Per the Solidity specification, a memory-safe block may only access the scratch space, temporary memory that is located after the value of the free memory pointer at the beginning of the assembly block, and memory ranges that are allocated via the free memory pointer or by Solidity. Despite this, no incorrect behavior has been observed in practice, since the data at `0x80` is dead by the time it is overwritten and `_erc1271Signer()` reads it back immediately.

Impact

The memory-safe annotation is incorrect per the Solidity specification. The Solidity optimizer is allowed to make assumptions about memory-safe blocks — for example, it may reorder or eliminate memory operations that it considers redundant. While no issues have been observed with the current compiler version, as of the time of writing, future optimizer improvements could introduce subtle bugs if they rely on the correctness of this annotation.

Recommendations

Consider using transient storage (`tstore` / `tload`) to pass the signer between `isValidSignature` and `_erc1271Signer` instead of a fixed memory slot. Transient storage operates in a separate address space from memory, eliminating the memory-safety violation entirely while preserving the ability to share state across the two functions without controlling the intermediate call flow through Solady's ERC-1271 implementation. However, this would require Solidity 0.8.24+ and EIP-1153 support on the target chain.

Alternatively, simply removing the memory-safe annotation would accurately reflect the block's behavior and prevent the optimizer from making incorrect assumptions about this code.

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [e6146d2c](#).

3.3. Unbounded timelock delay lacks on-chain safeguards

Target	DepositWalletFactory		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The `setTimelockDelay` function does not enforce any bounds on the timelock delay. While the admin role is behind a multi-sig and Polymarket confirmed this value would never intentionally be set to an unreasonable number, the function itself accepts any value without on-chain validation:

```
function setTimelockDelay(uint256 _timelockDelay) external onlyAdmin {
    _setTimelockDelay(_timelockDelay);
}

function _setTimelockDelay(uint256 _timelockDelay) internal {
    timelockDelay = _timelockDelay;

    emit TimelockDelaySet(_timelockDelay);
}
```

The timelock delay is read from the factory at runtime by every wallet's `onlyPaused` modifier:

```
modifier onlyPaused() {
    require(paused() != 0, NotPaused());
    require(
        block.timestamp > paused() + DepositWalletFactory(factory()).
            timelockDelay(),
        TimelockInsufficientDelay()
    );
    _;
}
```

A core invariant of the system is that no user action after off-chain matching should cause on-chain settlement to fail. The timelock is the mechanism that enforces this on chain — it prevents a wallet owner from pausing and immediately withdrawing funds or revoking approvals before the operator settles pending trades.

If the delay is set to zero, the timelock provides no protection; a wallet owner could front-run the

operator's `execute()` transaction by pausing their wallet and immediately transferring funds out or revoking approvals in the very next block. Although the protocol cancels existing orders off chain by monitoring Paused events, a zero delay means paused-only functions (such as `withdrawERC20` and `revokeAllowance`) can be called before the off-chain system has time to react. This allows the owner to drain funds or revoke approvals ahead of pending settlements, causing the inner calls within `execute()` to revert.

Conversely, an excessively large delay could lock users out of their own funds for an unreasonable period. Since the delay is read from the factory at runtime rather than snapshot when pausing, any change retroactively affects all currently paused wallets.

While this is operationally mitigated by the multi-sig setup, enforcing both a minimum and maximum bound on chain would serve as an inexpensive second layer of defense — guaranteeing a meaningful settlement window while preserving users' ability to withdraw within a reasonable time frame, regardless of admin configuration.

Impact

A value too low allows a wallet owner to front-run settlement by draining funds before the operator can react. A value too high locks users out of their own funds for an unreasonable period. Since the delay is global and read at runtime, either misconfiguration affects all deployed wallets simultaneously.

Recommendations

Enforce both a minimum and maximum bound on the timelock delay in `_setTimelockDelay`. A sensible minimum (e.g., a few hours, depending on operational needs) guarantees a meaningful reaction window for settlement at the contract level. An upper bound gives users on-chain guarantees that they can always withdraw their funds within a reasonable time frame.

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [817261ba](#).

3.4. Operator can invoke arbitrary contracts through unvalidated proxy calls

Target	DepositWalletFactory		
Category	Business Logic	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

The proxy function does not validate that `_batches[i].wallet` is an actual deposit wallet deployed by the factory. The operator can use it to call the `execute(Batch, bytes)` function selector on any contract address:

```
// src/DepositWalletFactory.sol#L204-L209
function proxy(Batch[] calldata _batches, bytes[] calldata _signatures)
    external onlyOperator {
    require(_batches.length == _signatures.length, ArrayLengthMismatch());
    for (uint256 i; i < _batches.length; ++i) {
        IDepositWallet(_batches[i].wallet).execute(_batches[i], _signatures[i]
    );
    }
}
```

The operator is a trusted role and transactions are generally submitted through a relayer, so this does not represent a direct security risk under the trust model as of the time of writing. However, it means the factory can be used as a proxy to interact with arbitrary contracts on behalf of the operator, which extends beyond its intended purpose.

Impact

Without on-chain validation, the proxy function's correctness is an off-chain guarantee rather than a contract-level invariant. A compromised or misconfigured relayer could use the factory's address as `msg.sender` to call arbitrary contracts. On-chain validation would close this gap and ensure the factory only interacts with wallets it has deployed.

Recommendations

Consider adding the initial implementation address to the immutable arguments. Therefore, the function proxy can calculate the expected wallet address with the immutable arguments and validate the provided wallet address before calling the function `execute`.

Remediation

This issue has been acknowledged by Polymarket.

Polymarket provided the following response to this finding:

We maintain an off-chain registry of deployed wallets (indexed from WalletDeployed events) and validate batch.wallet against it before submitting transactions. Doing the additional check will add gas since this is a frequently called function.

3.5. Missing zero-address check in wallet initialization allows ownerless wallets

Target	DepositWallet		
Category	Coding Mistakes	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

Neither the factory's deploy function nor the wallet's initialize and _initializeOwner functions validate that the owner address is not the zero address. The operator is a trusted role, and Polymarket confirmed that off-chain checks are in place to prevent this. However, address(0) is uniquely likely as an accidental value because it is the default zero value at the EVM level — if there is an unexpected bug in the relayer, a network issue, or a malformed request, missing or improperly encoded parameters will default to address(0), unlike address(1) or address(2), which would require an explicit and unusual input:

```
function deploy(address[] calldata _owners, bytes32[] calldata _ids)
    external onlyOperator {
    // [...]

    for (uint256 i; i < _owners.length; ++i) {
        // [...]

        IDepositWallet(wallet).initialize(_owners[i]);

        emit WalletDeployed(wallet, _owners[i], _ids[i], implementation_);
    }
}
```

The _initializeOwner function stores whatever value is passed without any validation:

```
// src/libraries/Ownable.sol#L61-L69
function _initializeOwner(address _newOwner) internal virtual {
    bytes32 slot = _OWNER_SLOT;

    assembly {
        sstore(slot, _newOwner)
    }

    emit OwnershipTransferred(address(0), _newOwner);
}
```

```
}
```

Note that `_transferOwnership` does validate against `address(0)`, but this check is absent from the initialization path. A wallet initialized with a zero-address owner would be permanently ownerless — no one could sign batches, authorize session signers, pause the wallet, or initiate ownership transfers. Any funds sent to the wallet would be irrecoverable since there is no valid signer to authorize these calls.

Impact

Each wallet deployed with `address(0)` as owner is permanently unrecoverable, and the protocol would need to redeploy a new wallet each time. Beyond the deployment itself, the impact compounds wasted gas from redeployments, potential state inconsistencies between on-chain wallets and off-chain mappings, and unexpected behavior in any downstream systems that reference the orphan wallet. Any funds deposited to the orphan wallet's deterministic address (which can be predicted before deployment) would be locked with no recovery path.

Recommendations

Add a `require(_owner != address(0))` check in `_initializeOwner` or in the `deploy` function. This is a single-line, almost zero-cost safeguard that catches the most common accidental misconfiguration before it becomes an on-chain problem.

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [a22a420e ↗](#).

3.6. Test suite

Target	Project-wide		
Category	Protocol Risks	Severity	Informational
Likelihood	N/A	Impact	Informational

Description

When building a complex contract with multiple moving parts (state changes) and dependencies, comprehensive testing is essential. This includes testing for both positive and negative scenarios on every function that touches the contract's state.

Positive tests should verify that each function's side effect is as expected, while negative tests should cover every revert, preferably in every logical branch.

Impact

The test coverage for this project could be expanded to include verification of events emitted by the function `execute`, which would facilitate off-chain monitoring.

Unit test cases that are missing include (but are not necessarily limited to) those documented in section [5.1](#).

Recommendations

We recommend expanding the test suite to ensure extensive and thorough test coverage.

Remediation

This issue has been acknowledged by Polymarket, and a fix was implemented in commit [e6146d2c](#).

4. Discussion

The purpose of this section is to document miscellaneous observations that we made during the assessment. These discussion notes are not necessarily security related and do not convey that we are suggesting a code change.

4.1. Cross-contract dependency on factory's sequential storage slots

The contract DepositWallet reads state variables directly from the factory at runtime. Specifically, timelockDelay gates all emergency recovery functions via the onlyPaused modifier, and authorizedImplementation gates wallet UUPS upgrades:

```
// src/DepositWallet.sol#L69-L76
modifier onlyPaused() {
    require(paused() != 0, NotPaused());
    require(
        block.timestamp > paused() + DepositWalletFactory(factory()).
            timelockDelay(),
        TimelockInsufficientDelay()
    );
    _;
}
```

```
// src/DepositWallet.sol#L371-L377
function _authorizeUpgrade(address _implementation)
    internal view override onlySelf {
    require(
        DepositWalletFactory(factory()).authorizedImplementation(_implementatio
            n),
        InvalidImplementation()
    );
}
```

The factory stores these variables in sequential storage slots (0, 1, 2):

```
// src/DepositWalletFactory.sol#L83-L90
address public implementation;           // slot 0
uint256 public timelockDelay;           // slot 1
mapping(address => bool) public authorizedImplementation; // slot 2
```

This contrasts with the contract DepositWallet, which uses computed named slots (keccak256("DepositWallet.*") - 1) specifically to avoid storage layout issues across

upgrades. If a factory upgrade were to introduce a new state variable above these declarations or reorder them, the slot positions would shift, and all deployed wallets' emergency withdrawal timing and upgrade authorization would silently break.

As of the time of writing, this is safe. The CI pipeline compares the storage layout against a committed snapshot (`.storage-layouts/DepositWalletFactory.md`), which is a good process-level safeguard. However, this can be bypassed by updating the snapshot alongside the code, and it does not provide the same technical guarantee that named slots offer.

Given the cross-contract dependency, aligning the factory on the same computed slot pattern as the wallet would provide defense in depth against accidental storage corruption during future upgrades.

This issue has been acknowledged by Polymarket.

Polymarket provided the following response to this finding:

We have a CI check for this.

4.2. Vulnerable wallet implementations cannot be forcibly retired on chain

When a wallet implementation is unauthorized via `unauthorizeImplementation`, this only prevents new deployments using that implementation and blocks upgrades to it. Existing wallets already running on the unauthorized implementation continue operating normally — they can still execute batches, authorize session signers, and perform all other functions:

```
// src/DepositWalletFactory.sol#L263-L267
function unauthorizeImplementation(address _implementation)
    external onlyOwner {
    authorizedImplementation[_implementation] = false;

    emit ImplementationUnauthorized(_implementation);
}
```

If a vulnerability is discovered in a deployed implementation, the factory owner can unauthorize it and authorize a patched version, but there is no on-chain mechanism to force wallets to upgrade or to pause wallets still running the vulnerable code. The pause function on each wallet is restricted to the wallet's own owner — the factory owner, admin, and operator cannot pause individual wallets.

The Polymarket team confirmed that this scenario would be handled off-chain; their systems would listen for relevant events, cancel all limit orders for affected wallets, stop accepting new orders, and stop relaying transactions for those wallets via the function proxy. Users would then be prompted to upgrade through the Polymarket interface. This approach is effective given Polymarket's architecture, where the operator is the sole entry point for batch execution via the

onlyFactory modifier.

This is worth noting as a design trade-off: the security of wallets on a vulnerable implementation depends entirely on the off-chain infrastructure ceasing to relay transactions. If the off-chain systems fail to act or if there is a delay in detection, wallets on the vulnerable implementation remain fully operational on chain.

This issue has been acknowledged by Polymarket.

Polymarket provided the following response to this finding:

It was a design decision to give wallet owners control of their deposit wallet.

4.3. Single batch failure in proxy reverts all batches in the transaction

The proxy function iterates over an array of batches and forwards each to its target wallet's execute function. If any single batch fails — due to an invalid signature, expired deadline, invalid nonce, or a failed call — the entire transaction reverts, and none of the batches execute:

```
// src/DepositWalletFactory.sol#L204-L209
function proxy(Batch[] calldata _batches, bytes[] calldata _signatures)
    external onlyOperator {
    require(_batches.length == _signatures.length, ArrayLengthMismatch());
    for (uint256 i; i < _batches.length; ++i) {
        IDepositWallet(_batches[i].wallet).execute(_batches[i], _signatures[i]
    );
    }
}
```

Each execute call reverts on any failure within the batch:

```
// src/DepositWallet.sol#L195-L204
for (; i < _batch.calls.length; i++) {
    // [...]
    (bool success, bytes memory result) =
        _batch.calls[i].target.call{value:
        _batch.calls[i].value}(_batch.calls[i].data);

    require(success, CallFailed());
    // [...]
}
```

If the operator were to bundle multiple users' batches into a single proxy call, an attacker could grief other users by submitting a batch designed to fail, causing all co-bundled batches to revert. This is a potential denial-of-service vector where one user's failing batch affects all other users in the same transaction.

The Polymarket team confirmed that they do not currently batch multiple users' operations into a single proxy call — each user's transaction is executed individually, giving a clean transaction hash per operation. The multiwallet batching functionality is present on chain as an option for future use.

If multiwallet batching is enabled in the future, wrapping each execute call in a try-catch block would allow other batches to succeed even if one fails, providing fault isolation between independent users' operations.

This issue has been acknowledged by Polymarket.

Polymarket provided the following response to this finding:

To give more context on how this is used in practice: we don't batch multiple users' operations into a single proxy() call. Each user/integrator txn is executed individually, which gives us (and third-party integrators) a clean txn hash per operation with the guarantee that if the txn succeeded, everything in it executed successfully.

The operator is fully trusted and controlled by us. It validates all executions via estimateGas before submitting, and decides what to include in each call. Since we're not bundling unrelated users' batches together, the griefing vector (one user's revert taking down another user's batch) doesn't apply to our current usage.

On the try/catch suggestion specifically. introducing that would break a useful invariant we rely on: a successful txn means full execution. With try/catch we'd need a separate indexing workflow to detect silently failed sub-calls, which adds complexity without addressing a real risk given our architecture. This complexity is not only to be handled by us but also third parties who rely on us to execute txns on their behalf.

The batching capability exists as built-in functionality and is reserved for future use. If and when we decide to enable multi-user batching, we'll revisit the error isolation question at that point.

5. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

5.1. Module: DepositWallet.sol

Function: `authorizeSessionSigner(address _sessionSigner, uint256 _validUntil)`

This function authorizes a session signer until a specified timestamp. It can only be called by the current contract itself.

Inputs

- `_sessionSigner`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The address to authorize as a session signer.
- `_validUntil`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The timestamp until which the session signer is authorized.

Branches and code coverage

Intended branches

- The `validUntil` slot of the specified session signer is updated to the provided timestamp.
 - ☒ Test coverage
- The event `SessionSignerAuthorized` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the contract itself.
 - ☒ Negative test

Function: `cancelOwnershipHandover()`

This function can cancel a pending ownership handover. It can only be called by the current contract itself.

Branches and code coverage

Intended branches

- The `_PENDING_OWNER_SLOT` is set to the zero address.
 - ☒ Test coverage
- The event `OwnershipHandoverCanceled` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the contract itself.
 - ☒ Negative test
- Reverts if there is no pending ownership handover to cancel.
 - ☒ Negative test

Function: `completeOwnershipHandover(uint256 _deadline, bytes _signature)`

This function completes a pending ownership transfer.

Inputs

- `_deadline`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must not be less than the current block timestamp.
 - **Impact:** The deadline included in the signed message.
- `_signature`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be a valid EIP-712 signature from the pending owner.
 - **Impact:** The EIP-712 signature from the pending owner that authorizes the ownership transfer.

Branches and code coverage

Intended branches

- The `_PENDING_OWNER_SLOT` is updated to the zero address.
 - ☒ Test coverage
- The `_OWNER_SLOT` is updated to the pending owner address.
 - ☒ Test coverage
- The event `OwnershipTransferred` is emitted.
 - ☒ Test coverage
- The event `OwnershipHandoverCompleted` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the `_deadline` has passed.
 - ☒ Negative test
- Reverts if there is no pending owner.
 - ☒ Negative test
- Reverts if the `_signature` is invalid.
 - ☒ Negative test

Function: `execute(Batch _batch, bytes _signature)`

This function can execute a signed batch of calls. The caller must be the deposit wallet factory.

Inputs

- `_batch`
 - **Control:** Fully controlled by the signer.
 - **Constraints:** The length of `_batch.calls` must be greater than zero. The calls can be arbitrary, but if the signature is from a session signer, calls targeting the wallet itself are not allowed. The `_batch.wallet` must be equal to the current contract address. The `_batch.nonce` must be equal to the nonce value stored in the contract. The `_batch.deadline` must be in the future.
 - **Impact:** The batch containing the target wallet, nonce, deadline, and calls to execute.
- `_signature`
 - **Control:** Fully controlled by the caller.

- **Constraints:** Must be a valid EIP-712 signature from the owner or an authorized session signer.
- **Impact:** Authorizes the execution of the batch.

Branches and code coverage

Intended branches

- The call targeting the wallet itself can be successfully executed when the signer is the owner.
☒ Test coverage
- The call targeting the other contracts can be successfully executed when the signer is an authorized session signer.
☒ Test coverage
- The call targeting the other contracts can be successfully executed when the signer is the owner.
☒ Test coverage
- The event Execution is emitted for each call in the batch with the target, value, calldata, and result.
☐ Test coverage
- The event BatchExecuted is emitted with the executed nonce after successful execution of the batch.
☐ Test coverage
- The stored nonce value increases by one after execution.
☒ Test coverage

Negative behavior

- Reverts if the caller is not the factory.
☒ Negative test
- Reverts if the `_batch.calls` array is empty.
☒ Negative test
- Reverts if the `_batch.wallet` does not match the current contract address.
☒ Negative test
- Reverts if the `_batch.nonce` does not match the expected nonce.
☒ Negative test
- Reverts if the `_batch.deadline` has passed.
☒ Negative test

- Reverts if the signer extracted from the `_signature` is not the contract owner or an authorized session signer.
 - ☒ Negative test
- Reverts if the signature is invalid.
 - ☒ Negative test
- Reverts if the signer is the zero address.
 - ☒ Negative test
- Reverts if the signer extracted from the `_signature` does not match the result of `ecrecover` on the batch digest.
 - ☒ Negative test
- Reverts if a session signer attempts to execute a call targeting the wallet itself.
 - ☒ Negative test
- Reverts if any call in the batch fails.
 - ☒ Negative test

Function: `pause()`

This function allows the owner to set the `_PAUSED_SLOT` to the current block timestamp.

Branches and code coverage

Intended branches

- The `_PAUSED_SLOT` is set to the current block timestamp.
 - ☒ Test coverage
- The event `Paused` is emitted with the paused timestamp.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.
 - ☒ Negative test

Function: `revokeAllowance(address _token, address _spender)`

This function allows the owner to revoke a wallet's ERC-20 allowance. The wallet must be paused and the timelock delay must have elapsed before calling this function.

Inputs

- `_token`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The ERC-20 token address.
- `_spender`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The spender whose allowance is revoked.

Branches and code coverage

Intended branches

- The allowance for the specified spender is set to zero.
 - ☒ Test coverage
- The event `AllowanceRevoked` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.
 - ☒ Negative test
- Reverts if the wallet is not currently paused.
 - ☒ Negative test
- Reverts if the timelock delay has not yet elapsed.
 - ☒ Negative test

Function: `revokeApprovalForAll(address _token, address _operator)`

This function allows the owner to revoke an ERC-1155 operator approval. The wallet must be paused and the timelock delay must have elapsed before calling this function.

Inputs

- `_token`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.

- **Impact:** The ERC-1155 token address.
- `_operator`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The operator whose approval is revoked.

Branches and code coverage

Intended branches

- The operator approval is revoked.
 - ☒ Test coverage
- The event `ApprovalForAllRevoked` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.
 - ☒ Negative test
- Reverts if the wallet is not currently paused.
 - ☒ Negative test
- Reverts if the timelock delay has not yet elapsed.
 - ☒ Negative test

Function: `revokeSessionSigner(address _sessionSigner)`

This function revokes a session signer's authorization. It can only be called by the current contract itself.

Inputs

- `_sessionSigner`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The session signer address to revoke.

Branches and code coverage

Intended branches

- The `validUntil` slot of the specified session signer is set to zero.

☒ Test coverage

- The event `SessionSignerRevoked` is emitted.

☒ Test coverage

Negative behavior

- Reverts if the caller is not the contract itself.

☒ Negative test

Function: `revokeSessionSignerEmergency(address _sessionSigner)`

This function allows the owner to directly revoke a session signer. The wallet must be paused and the timelock delay must have elapsed before calling this function.

Inputs

- `_sessionSigner`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The session signer to be revoked.

Branches and code coverage

Intended branches

- The `validUntil` slot of the specified session signer is set to zero.

☒ Test coverage

- The event `SessionSignerRevokedEmergency` is emitted.

☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.

☒ Negative test

- Reverts if the wallet is not currently paused.

☒ Negative test

- Reverts if the timelock delay has not yet elapsed.

☒ Negative test

Function: transferOwnership(address _newOwner)

This function initiates a two-step ownership transfer to a new owner. It can only be called by the current contract itself.

Inputs

- `_newOwner`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must not be the zero address.
 - **Impact:** The proposed new owner address.

Branches and code coverage

Intended branches

- The `_PENDING_OWNER_SLOT` is set to the `_newOwner` address.
 - ☒ Test coverage
- The event `OwnershipHandoverRequested` is emitted with the `_newOwner` address.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the contract itself.
 - ☒ Negative test
- Reverts if the `_newOwner` is the zero address.
 - ☒ Negative test

Function: unpause()

This function allows the owner to set the `_PAUSED_SLOT` to zero. The wallet must be currently paused and the timelock delay must have elapsed before calling this function.

Branches and code coverage

Intended branches

- The `_PAUSED_SLOT` is set to zero.
 - ☒ Test coverage
- The event `Unpaused` is emitted with the unpaused timestamp.

☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.

☒ Negative test

- Reverts if the wallet is not currently paused.

☒ Negative test

- Reverts if the timelock delay has not yet elapsed.

☒ Negative test

Function: `upgrade(address _implementation, bytes _data)`

This function upgrades the wallet implementation via UUPS. It can only be called by the current contract itself. The contract upgrade is also capable of calling the function `upgradeToAndCall` in the parent contract `UUPSUpgradeable`.

Inputs

- `_implementation`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be an implementation address authorized by the factory.
 - **Impact:** The new implementation contract address.
- `_data`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** Optional calldata to execute on the new implementation after upgrade.

Branches and code coverage

Intended branches

- The proxy's implementation is upgraded to the new `_implementation` address.

☒ Test coverage

Negative behavior

- Reverts if the caller is not the contract itself.

☒ Negative test

- Reverts if the `_implementation` is not authorized by the factory.

☒ Negative test

Function: `withdrawERC20(address _token, address _to, uint256 _amount)`

This function allows the owner to withdraw ERC-20 tokens from the paused wallet after the timelock delay has elapsed.

Inputs

- `_token`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The ERC-20 token address to withdraw.
- `_to`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The recipient address.
- `_amount`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The amount of tokens to withdraw.

Branches and code coverage

Intended branches

- The specified amount of ERC-20 tokens is transferred from the wallet to the recipient address.
 - ☒ Test coverage
- The event `ERC20Withdrawal` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.
 - ☒ Negative test
- Reverts if the wallet is not currently paused.

- ☒ Negative test
- Reverts if the timelock delay has not yet elapsed.
- ☒ Negative test

Function: `withdrawERC1155(address _token, address _to, uint256[] _ids, uint256[] _amounts)`

This function allows the owner to batch-withdraw ERC-1155 tokens from the paused wallet after the timelock delay has elapsed.

Inputs

- `_token`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The ERC-1155 token address to withdraw.
- `_to`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The recipient address.
- `_ids`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The token IDs to withdraw.
- `_amounts`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The amounts for each token ID.

Branches and code coverage

Intended branches

- The ERC-1155 tokens are transferred from the wallet to the recipient address.
 - ☒ Test coverage
- The event `ERC1155BatchWithdrawal` is emitted.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner.
 - ☒ Negative test
- Reverts if the wallet is not currently paused.
 - ☒ Negative test
- Reverts if the timelock delay has not yet elapsed.
 - ☒ Negative test

5.2. Module: DepositWalletFactory.sol

Function: addAdmin(address _admin)

This function allows the owner of the contract to grant the admin role to another address. The owner can also grant the admin role to an address through the function grantRoles of the contract OwnableRoles.

Inputs

- _admin
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The address to grant the admin role to.

Branches and code coverage

Intended branches

- The admin role is successfully granted to the specified address.
 - ☒ Test coverage
- The other roles of the specified address remain unchanged.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner of the contract.
 - ☒ Negative test

Function: addOperator(address _operator)

This function allows an account with the admin role to grant the operator role to another address. The owner of the contract can also grant the operator role to an address through the function grantRoles of the contract OwnableRoles.

Inputs

- `_operator`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The address to grant the operator role to.

Branches and code coverage**Intended branches**

- The operator role is successfully granted to the specified address.
 - ☒ Test coverage
- The other roles of the specified address remain unchanged.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role.
 - ☒ Negative test

Function: authorizeImplementation(address _implementation)

This function allows the owner of the contract to authorize an implementation address, which can be used for wallet deployments or UUPS upgrades.

Inputs

- `_implementation`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must not be the zero address.
 - **Impact:** The implementation address to authorize.

Branches and code coverage

Intended branches

- The implementation address is successfully authorized.
 - ☒ Test coverage
- The event `ImplementationAuthorized` is emitted after the implementation is authorized.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner of the contract.
 - ☒ Negative test
- Reverts if the provided implementation address is the zero address.
 - ☒ Negative test

Function: `deploy(address[] _owners, byte[32][] _ids)`

This function allows an account with the operator role to deploy multiple deposit wallets in a single transaction. Each wallet is deployed as a deterministic ERC-1967 proxy using the current default implementation.

Inputs

- `_owners`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** The length of the `_owners` array must be equal to the length of the `_ids` array.
 - **Impact:** Each address in the `_owners` array will be set as the initial owner of a newly deployed wallet.
- `_ids`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** The length of the `_ids` array must be equal to the length of the `_owners` array.
 - **Impact:** The unique identifiers for each wallet. This will affect the calculation of the wallet address.

Branches and code coverage

Intended branches

- The deposit wallet can be successfully deployed.
 - ☑ Test coverage
- The wallet is deployed at the predicted deterministic address.
 - ☑ Test coverage
- The event `WalletDeployed` is emitted after each wallet deployment.
 - ☑ Test coverage

Negative behavior

- Reverts if the caller does not have the operator role.
 - ☑ Negative test
- Reverts if the length of the `_owners` array is not equal to the length of the `_ids` array.
 - ☑ Negative test
- Reverts if the current default implementation is not authorized.
 - ☑ Negative test
- Reverts if the target address has already deployed a contract.
 - ☑ Negative test

Function: `proxy(Batch[] _batches, bytes[] _signatures)`

This function allows an account with the operator role to call the function `execute` on multiple deposit wallets through the factory contract.

Inputs

- `_batches`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** The length of the `_batches` array must be equal to the length of the `_signatures` array. Please refer to the `execute` function (in section [5.1](#), ↗) for more details on the constraints of each batch.
 - **Impact:** Each batch contains the target wallet address, the nonce, the execution deadline, and the list of calls to execute on that wallet.
- `_signatures`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** The length of the `_signatures` array must be equal to the length of the `_batches` array. Please refer to the `execute` function (in section [5.1](#), ↗) for more details on the constraints of each signature.
 - **Impact:** Each signature will be used to authorize the execution of the

corresponding batch.

Branches and code coverage

Intended branches

- Each batch can be successfully executed on the target wallet.

☒ Test coverage

Negative behavior

- Reverts if the caller does not have the operator role.

☒ Negative test

- Reverts if the length of the `_batches` array is not equal to the length of the `_signatures` array.

☒ Negative test

- Reverts if any batch fails its execution on the target wallet.

☒ Negative test

Function: `removeAdmin(address _admin)`

This function allows the owner of the contract to remove the admin role from another address. The owner can also remove the admin role from an address through the function `revokeRoles` of the contract `OwnableRoles`.

Inputs

- `_admin`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The address to remove the admin role from.

Branches and code coverage

Intended branches

- The admin role is successfully removed from the specified address.

☒ Test coverage

- The other roles of the specified address remain unchanged.

☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner of the contract.

☐ Negative test

Function: `removeOperator(address _operator)`

This function allows an account with the admin role to remove the operator role from another address. The owner of the contract can also remove the operator role from an address through the function `revokeRoles` of the contract `OwnableRoles`.

Inputs

- `_operator`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The address to remove the operator role from.

Branches and code coverage

Intended branches

- The operator role is successfully removed from the specified address.

☒ Test coverage

- The other roles of the specified address remain unchanged.

☒ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role.

☒ Negative test

Function: `setImplementation(address _implementation)`

This function allows the owner of the contract to set the default wallet implementation for new deployments.

Inputs

- `_implementation`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** Must be an authorized implementation address.
 - **Impact:** The new default implementation address of the deposit wallet.

Branches and code coverage

Intended branches

- The state variable `implementation` is successfully updated to the new implementation address.
 - ☒ Test coverage
- The event `ImplementationSet` is emitted after the implementation is updated.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner of the contract.
 - ☒ Negative test
- Reverts if the provided implementation address is not authorized.
 - ☒ Negative test

Function: `setTimelockDelay(uint256 _timelockDelay)`

This function allows an account with the admin role to set the timelock delay for paused wallet operations. The timelock delay is the amount of time that must elapse after a wallet is paused before the owner can execute paused-only operations on that wallet.

Inputs

- `_timelockDelay`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The new timelock delay in seconds.

Branches and code coverage

Intended branches

- The timelock delay is successfully updated.
 - ☒ Test coverage
- The event `TimeLockDelaySet` is emitted after the timelock delay is updated.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller does not have the admin role.
 - ☒ Negative test

Function: `unauthorizeImplementation(address _implementation)`

This function allows the owner of the contract to unauthorize an implementation address, which will prevent it from being used for future wallet deployments or UUPS upgrades.

Inputs

- `_implementation`
 - **Control:** Fully controlled by the caller.
 - **Constraints:** N/A.
 - **Impact:** The implementation address to unauthorize.

Branches and code coverage

Intended branches

- The implementation address is successfully unauthorized.
 - ☒ Test coverage
- The event `ImplementationUnauthorized` is emitted after the implementation is unauthorized.
 - ☒ Test coverage

Negative behavior

- Reverts if the caller is not the owner of the contract.
 - ☒ Negative test

6. Assessment Results

During our assessment on the scoped Deposit Wallet contracts, we discovered six findings. No critical issues were found. Two findings were of low impact and the remaining findings were informational in nature.

6.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.